

RESUMO DO MÓDULO

Servindo Arquivos Estáticos

No Express.js, é possível servir arquivos estáticos, como HTML, CSS, JavaScript, imagens e outros tipos de mídia, usando o middleware `express.static()`. Esses tipos de arquivos são chamados de arquivos estáticos porque não exigem nenhuma alteração ou processamento dinâmico enquanto o usuário usa o site.

O Que São Arquivos Estáticos?

Arquivos estáticos são arquivos que não mudam em tempo de execução. Exemplos comuns incluem:

- Arquivos HTML
- Folhas de estilo CSS
- Arquivos JavaScript
- Imagens (PNG, JPG, GIF)
- Vídeos e áudios

Esses arquivos geralmente são pré-carregados no servidor e servidos diretamente aos clientes que os solicitam, o que torna o carregamento mais rápido e eficiente.

Usando o Middleware `express.static()`

Para servir arquivos estáticos com o Express.js, usamos o middleware `express.static()`. Esse middleware serve automaticamente arquivos de um diretório especificado.

A sintaxe é:

```
express.static(root, [options])
```

- root: O caminho do diretório que contém os arquivos estáticos.

- options: Parâmetros adicionais para configurar o comportamento do middleware (opcional).

Vamos ver como configurar e usar o middleware `express.static()` na prática.

Exemplo de Uso Básico

Primeiro, vamos criar um diretório chamado `public` e adicionar um arquivo `index.html` dentro dele.

Em seguida, criaremos um servidor Express para servir os arquivos desse diretório:

```
const express = require('express');

const app = express();

// Serve arquivos estáticos da pasta "public"
app.use(express.static('./public'));

// Inicializa o servidor
app.listen(3000, () => {
  console.log('Servidor iniciado em http://localhost:3000');
});
```

Explicação do Código

- `app.use(express.static('./public'))`: Define que todos os arquivos dentro do diretório `public` estarão disponíveis para serem acessados através da URL base. Se o arquivo `index.html` estiver dentro desse diretório, ele será automaticamente carregado quando acessarmos `http://localhost:3000`.

Você pode adicionar mais arquivos dentro de `public`, como `contato.html`. Para acessá-lo, basta digitar `http://localhost:3000/contato.html`. Não é necessário incluir o nome da pasta (`public`) na URL, pois o Express já mapeia isso automaticamente.

Servindo Vários Diretórios Estáticos

É possível definir vários diretórios para servir arquivos estáticos. Vamos adicionar um diretório chamado files e um GIF de exemplo para ver como isso funciona:

```
const express = require('express');

const app = express();

// Serve arquivos estáticos da pasta "public"
app.use(express.static('./public'));

// Serve arquivos estáticos da pasta "files"
app.use(express.static('./files'));

// Inicializa o servidor
app.listen(3000, () => {
  console.log('Servidor iniciado em http://localhost:3000');
});
```

Agora, se você acessar <http://localhost:3000/gato.gif> (assumindo que gato.gif esteja dentro da pasta files), ele será carregado diretamente.

Especificando um Caminho de URL Personalizado

Você pode especificar um caminho de URL diferente para servir os arquivos. Por exemplo, se quiser que os arquivos sejam acessíveis em <http://localhost:3000/assets> em vez de <http://localhost:3000>, faça o seguinte:

```
app.use('/assets', express.static('./public'));
```

Agora, os arquivos dentro de public estarão disponíveis em <http://localhost:3000/assets>.

Usando `path.join()` para Definir o Caminho dos Arquivos

Quando o caminho do diretório não está claro ou depende do ambiente, é recomendável usar o módulo `path` do Node.js para definir o caminho absoluto do diretório:

```
const express = require('express');
```

```
const path = require('path');
```

```
const app = express();
```

```
// Define o caminho absoluto para o diretório "public"
```

```
const publicDirectoryPath = path.join(__dirname, 'public');
```

```
// Serve arquivos estáticos da pasta "public"
```

```
app.use(express.static(publicDirectoryPath));
```

```
// Inicializa o servidor
```

```
app.listen(3000, () => {
```

```
  console.log('Servidor iniciado em http://localhost:3000');
```

```
});
```

Explicação do Código

- `path.join(__dirname, 'public')`: Concatena o caminho absoluto do diretório atual (`__dirname`) com o diretório `public`.
- Isso garante que o Express sempre servirá os arquivos estáticos do diretório `public`,

independentemente de onde o aplicativo for executado.

Servindo Arquivos Estáticos de Maneira Mais Segura

Ao definir diretórios estáticos, tenha cuidado para não expor diretórios que contenham informações sensíveis ou arquivos que você não deseja que sejam públicos. Defina apenas diretórios que contenham arquivos destinados a serem servidos aos usuários.

Conclusão

Servir arquivos estáticos com o Express.js é uma maneira simples e eficiente de lidar com conteúdo que não precisa ser gerado dinamicamente. Usar o middleware `express.static()` permite que você organize seus recursos estáticos (HTML, CSS, JavaScript, imagens) e os sirva de forma rápida e otimizada.

Além disso, ao combinar `express.static()` com o uso de caminhos relativos (`path.join()`), você garante que seu aplicativo seja portátil e fácil de configurar em diferentes ambientes. Isso torna a gestão de arquivos estáticos no Express simples, robusta e eficiente.
